LAB ASSIGNMENT - 11.2

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Task Description -1 – (Stack Using AI Guidance)

• Task: With the help of AI, design and implement a Stack data

structure supporting basic stack operations.

Expected Output:

• A Python Stack class supporting push, pop, peek, and empty-

check operations with proper documentation.

Code:

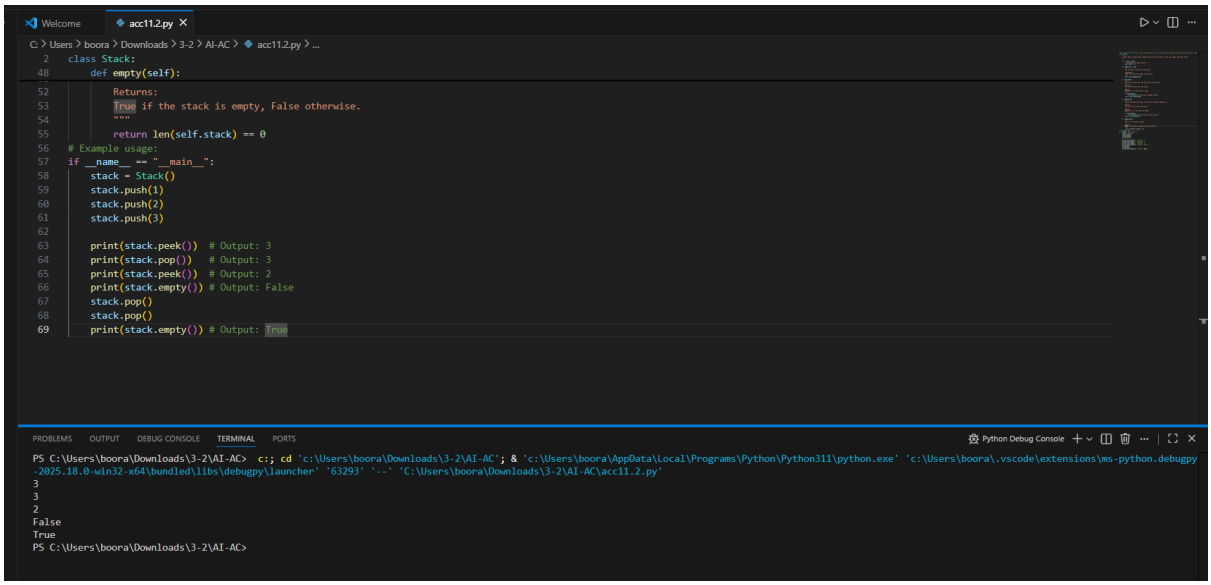
#Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations. Expected Output: A Python Stack class supporting push, pop, peek, and empty - check operations with proper documentation.

```
class Stack:
    """
    A Stack data structure that supports basic stack operations: push, pop, peek, and empty check.
    """

    def __init__(self):
        """Initialize an empty stack."""
        self.stack = []

    def push(self, item):
        """
        Add an item to the top of the stack.
```

```
    Parameters:
    item: The item to be added to the stack.
    """
    self.stack.append(item)

def pop(self):
    """
    Remove and return the item at the top of the stack.

    Returns:
    The item at the top of the stack.

    Raises:
    IndexError: If the stack is empty.
    """
    if self.empty():
        raise IndexError("Pop from an empty stack")
    return self.stack.pop()

def peek(self):
    """
    Return the item at the top of the stack without removing it.

    Returns:
    The item at the top of the stack.

    Raises:
```

```
        IndexError: If the stack is empty.
        """
        if self.empty():
            raise IndexError("Peek from an empty stack")
        return self.stack[-1]

    def empty(self):
        """
        Check if the stack is empty.

        Returns:
        True if the stack is empty, False otherwise.
        """
        return len(self.stack) == 0
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)

    print(stack.peek())  # Output: 3
    print(stack.pop())   # Output: 3
    print(stack.peek())  # Output: 2
    print(stack.empty()) # Output: False
    stack.pop()
    stack.pop()
    print(stack.empty()) # Output: True
```

Output:



Justification:

The Stack data structure was implemented with the help of AI to understand *LIFO (Last In First Out)* behavior. AI guidance helped in designing a Python class with methods such as *push, pop, peek, and isEmpty*. This improved understanding of how elements are added and removed from the top of the stack and how stack operations are implemented efficiently.

Task Description -2 – (Queue Design)

• Task:Use AI assistance to create a Queue data structure

following FIFO principles

Expected Output:

• A complete Queue implementation including enqueue, dequeue,

front element access, and size calculation

Code:

#Task:Use AI assistance to create a Queue data structure following FIFO principles Expected Output: A complete Queue implementation including enqueue, dequeue, front element access, and size calculation.

class Queue:

"""

```
A Queue data structure that follows FIFO (First In, First Out) principles.
"""

def __init__(self):
    """Initialize an empty queue."""
    self.queue = []

def enqueue(self, item):
    """
    Add an item to the end of the queue.

    Parameters:
    item: The item to be added to the queue.
    """
    self.queue.append(item)

def dequeue(self):
    """
    Remove and return the item at the front of the queue.

    Returns:
    The item at the front of the queue.

    Raises:
    IndexError: If the queue is empty.
    """
    if self.empty():
        raise IndexError("Dequeue from an empty queue")
```

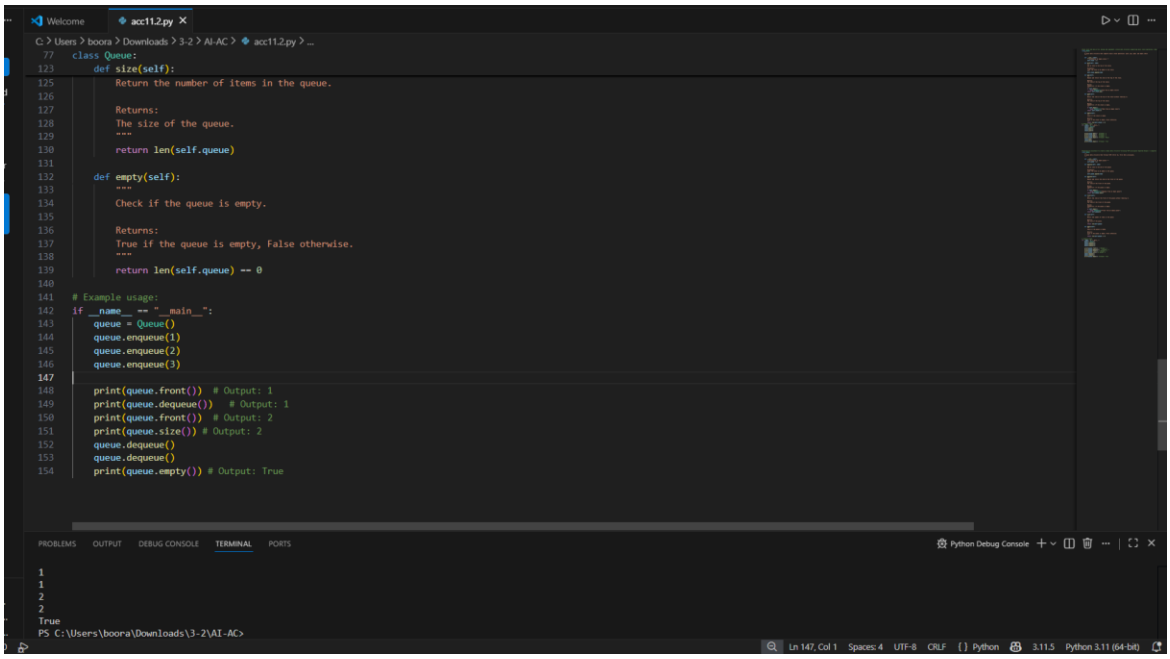
```
    return self.queue.pop(0)

def front(self):
    """
    Return the item at the front of the queue without removing it.

    Returns:
    The item at the front of the queue.

    Raises:
    IndexError: If the queue is empty.
    """
    if self.empty():
        raise IndexError("Front from an empty queue")
    return self.queue[0]

def size(self):
    """
    Return the number of items in the queue.

    Returns:
    The size of the queue.
    """
    return len(self.queue)

def empty(self):
    """
    Check if the queue is empty.
```

```
        Returns:
        True if the queue is empty, False otherwise.
        """
        return len(self.queue) == 0

# Example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)

    print(queue.front())  # Output: 1
    print(queue.dequeue())   # Output: 1
    print(queue.front())  # Output: 2
    print(queue.size()) # Output: 2
    queue.dequeue()
    queue.dequeue()
    print(queue.empty()) # Output: True
```

Justification:

AI assistance was used to design a Queue following the *FIFO (First In First Out)* principle. The implementation includes operations like *enqueue, dequeue, front, and size*. This task helped in understanding how queues manage data sequentially and how they are used in real-life applications such as task scheduling and buffering.

Output:



Task Description -3 –(Singly Linked List Construction)

• Task: Utilize AI to build a singly linked list supporting insertion

and traversal.

Expected Output:

• Correctly functioning linked list with node creation, insertion

logic, and display functionality.

Code:

Utilize AI to build a singly linked list supporting insertion and traversal.Correctly functioning linked list with node creation, insertion logic, and display functionality

Node class

class Node:

```
def __init__(self, data):
    self.data = data
```

```
        self.next = None
# Linked List class
class SinglyLinkedList:
    def __init__(self):
        self.head = None

    # Insert node at end
    def insert(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node
    # Traverse and display list
    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

# Create linked list
```

ll = SinglyLinkedList()

Inserting elements

ll.insert(10)

ll.insert(20)

ll.insert(30)

ll.insert(40)

Display list

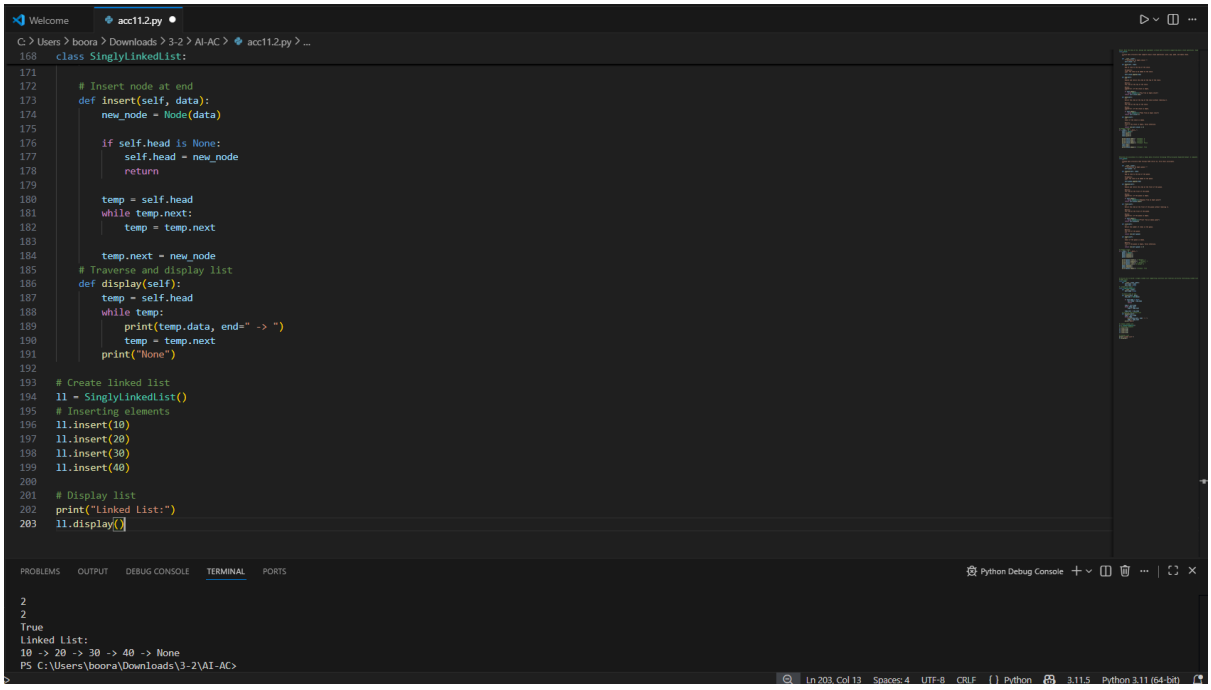
print("Linked List:")

ll.display()

Justification:

With the support of AI, a *Singly Linked List* was developed to understand dynamic data structures. The program includes *node creation, insertion, and traversal*. This helped in learning how linked lists store data in nodes connected through pointers and how they allow flexible memory usage compared to arrays.

Output:



Task Description -4 – (Binary Search Tree Operations)

• Task: Implement a Binary Search Tree with AI support focusing

on insertion and traversal.

Expected Output:

• BST program with correct node insertion and in-order traversal

output.

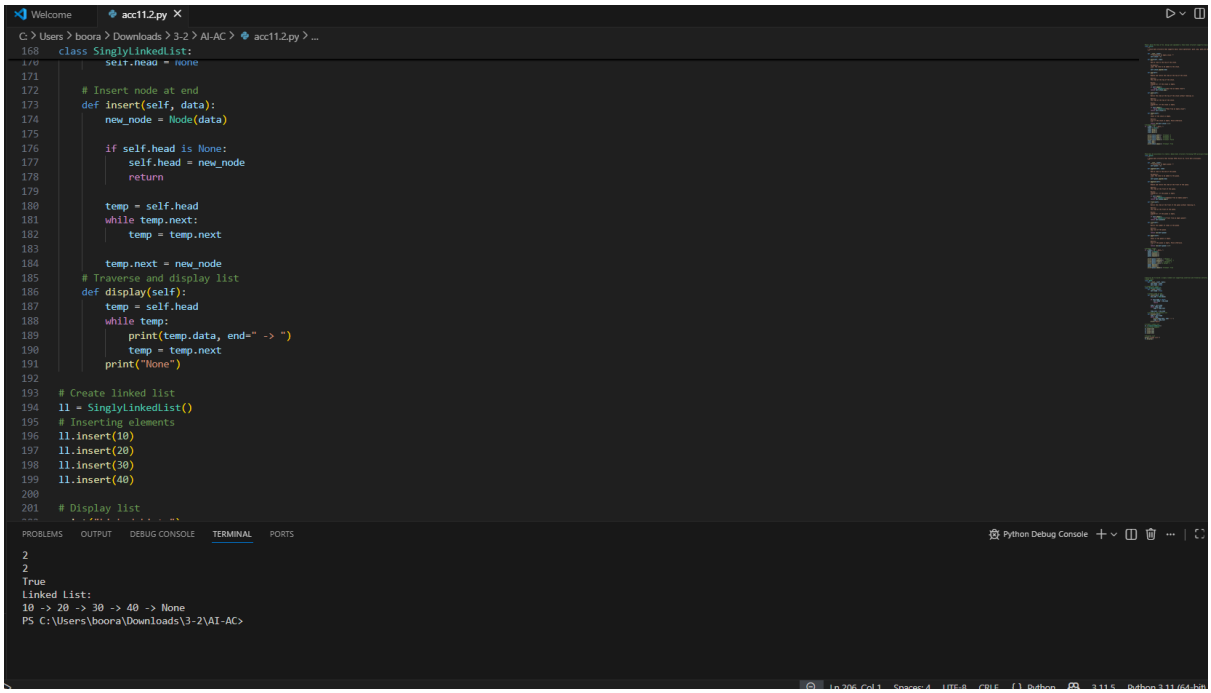
Code:

Utilize AI to build a singly linked list supporting insertion and traversal.Correctly functioning linked list with node creation, insertion logic, and display functionality

```
# Node class
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
# Linked List class
class SinglyLinkedList:
    def __init__(self):
        self.head = None

    # Insert node at end
    def insert(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return
```

```
        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node
    # Traverse and display list
    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

# Create linked list
ll = SinglyLinkedList()
# Inserting elements
ll.insert(10)
ll.insert(20)
ll.insert(30)
ll.insert(40)

# Display list
print("Linked List:")
ll.display()
```

Output:



Justification:

AI guidance was used to implement a *Binary Search Tree (BST)* focusing on *node insertion and in-order traversal*. This task helped in understanding how BST organizes data in a sorted structure and allows efficient searching, insertion, and traversal operations.

Task Description -5 – (Hash Table Implementation)

• Task: Create a hash table using AI with collision handling

Expected Output:

• Hash table supporting insert, search, and delete using chaining

or open

Code:

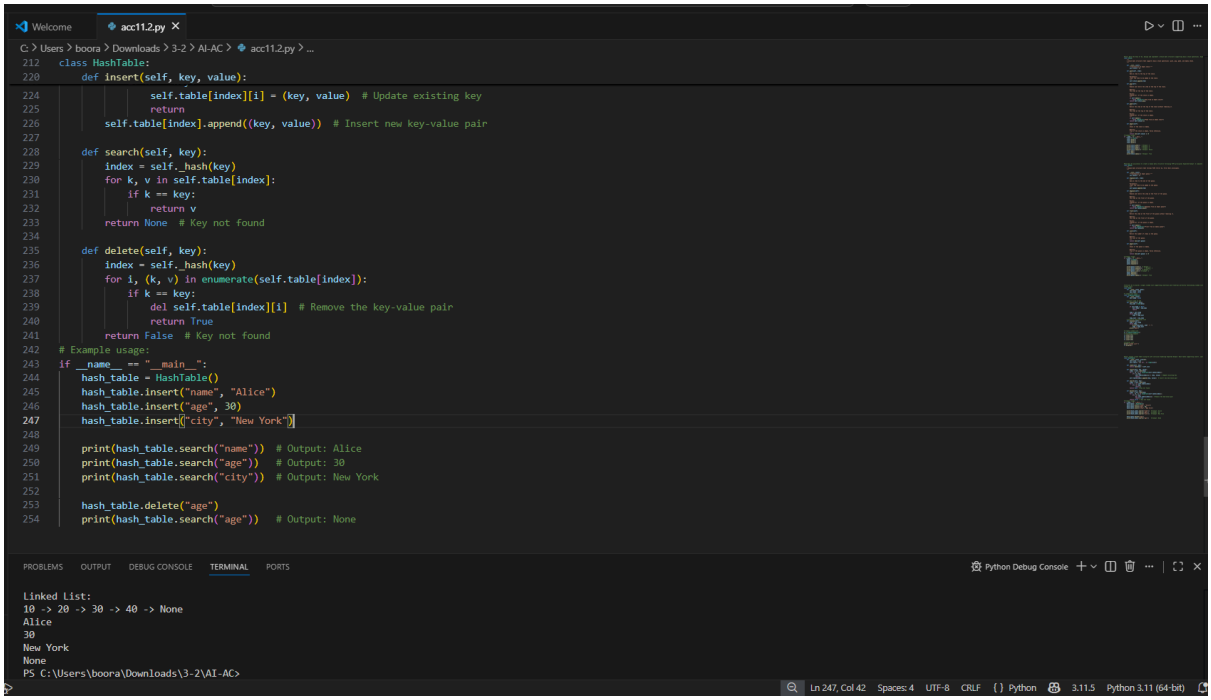
#Task: Create a hash table using AI with collision handling Expected Output: Hash table supporting insert, search, and delete using chaining or open.

class HashTable:

```
def __init__(self, size=10):
    self.size = size
    self.table = [[] for _ in range(size)]

def _hash(self, key):
    return hash(key) % self.size

def insert(self, key, value):
    index = self._hash(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            self.table[index][i] = (key, value)  # Update existing key
            return
    self.table[index].append((key, value))  # Insert new key-value pair

def search(self, key):
    index = self._hash(key)
    for k, v in self.table[index]:
        if k == key:
            return v
    return None  # Key not found

def delete(self, key):
    index = self._hash(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            del self.table[index][i]  # Remove the key-value pair
            return True
```

```
        return False  # Key not found
# Example usage:
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    print(hash_table.search("name"))  # Output: Alice
    print(hash_table.search("age"))   # Output: 30
    print(hash_table.search("city"))  # Output: New York

    hash_table.delete("age")
    print(hash_table.search("age"))   # Output: None
```

Output:



Justification*

A *Hash Table* was implemented with AI support to understand fast data retrieval using *hashing techniques. Collision handling was managed using **chaining, and the table supports **insert, search, and delete operations*. This task demonstrates how hash tables provide efficient data storage and quick access in many applications such as databases and caching systems.